

ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук , доцент
должность, уч. степень, звание

подпись, дата

А. А. Попов
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

по курсу: Программирование встроенных приложений

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4131з		М.Д. Быстров
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Цель работы

Привитие практических навыков по работе с системой прерываний ядра ARMv7-M и технической документацией.

Задание на лабораторную работу

Данные для варианта 4:

№ вар.	Номера линий РС	Количество групп и подгрупп приоритетов		Номера приоритетов соответственно порядку линий, №группы(№подгруппы)	Номер таймера - приоритет	Период обновления таймера, сек
		гр.	подгр.			
4	0,1,3,4	4	4	2(0), 3(2), 1(3), 2(2)	TIM4-3(1)	1

Листинг программы

```
#include "RTE_Components.h" // Component selection
#include CMSIS_device_header // Device header
#include <stdio.h>

static volatile uint32_t ui_count100ms=0;
void delay(void){
    volatile uint32_t i=6000000;
    while(i > 0)
        i--;
}

//int fputc(int c, FILE *f) { return ITM_SendChar(c); }

void SetupTIM4() {
    // enable TIM4 clock
    RCC->APB1ENR |= RCC_APB1ENR_TIM4EN;

    ITM_SendChar('\n');

    // reset configuration
    TIM4->CR1 = 0;

    // setup prescaler and autoreset
    TIM4->PSC = 7200 - 1;
    TIM4->ARR = 10000 - 1;

    // event timer updated
    TIM4->EGR |= TIM_EGR_UG;

    // enable update interrupt
    TIM4->DIER |= TIM_DIER_UIE;

    // set priority for timer interrupt
    NVIC_SetPriority(TIM4_IRQn, NVIC_EncodePriority(NVIC_GetPriorityGrouping(), 3, 1));
    NVIC_EnableIRQ(TIM4_IRQn);

    // TIM_CR1_CEN - Counter Enable
```

```

// TIM_CR1_ARPE - Auto-reload preload enable
TIM4->CR1 |= (TIM_CR1_CEN | TIM_CR1_ARPE);
}

int main(void) {
    uint32_t priGroup = 0, PreemptPriority=0, SubPriority=0;
    // enable HSE
    SET_BIT(RCC -> CR, RCC_CR_HSEON);
    while((RCC->CR & RCC_CR_HSERDY)==0){}
    // setup commands prefetch
    FLASH->ACR = FLASH_ACR_PRFTBE|FLASH_ACR_LATENCY_1;

    // 8 * 9 = 72 in PLL
    RCC->CFGR |= (uint32_t)(RCC_CFGR_PLLSRC_HSE_PREDIV |
RCC_CFGR_PLLMUL9);
    SET_BIT(RCC -> CR, RCC_CR_PLLON);
    while((RCC->CR & RCC_CR_PLLRDY) == 0){}

    // SYSCLK from PLL
    RCC->CFGR &= (uint32_t)((uint32_t)~(RCC_CFGR_SW));
    RCC->CFGR |= (uint32_t)RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) !=
(uint32_t)RCC_CFGR_SWS_PLL){}

    SystemCoreClockUpdate();//get SystemCoreClock
    printf("clk=%d\n", SystemCoreClock);

    //enable SYSCFG clockx
    SET_BIT(RCC -> APB2ENR, RCC_APB2ENR_SYSCFGEN);

    //clock GPIOC
    SET_BIT(RCC -> AHBENR, RCC_AHBENR_GPIOCEN);

    //PC0,1,3,4 In
    CLEAR_BIT(
        GPIOC->MODER,

        GPIO_MODER_MODER0|GPIO_MODER_MODER1|GPIO_MODER_MODER3|GPIO_M
ODER_MODER4);

    //Pull up PC0,1,3,4
    SET_BIT(
        GPIOC->PUPDR,

        GPIO_PUPDR_PUPDR0_0|GPIO_PUPDR_PUPDR1_0|GPIO_PUPDR_PUPDR3_0|GPIO_
PUPDR_PUPDR4_0);

    //PC2,5,6,7 Out
    SET_BIT(
        GPIOC->MODER,

        GPIO_MODER_MODER2_0|GPIO_MODER_MODER5_0|GPIO_MODER_MODER6_0|G
PIO_MODER_MODER7_0);

    // open stock mode

```

```

SET_BIT(
    GPIOC->OTYPER,

GPIO_OTYPER_OT_2|GPIO_OTYPER_OT_5|GPIO_OTYPER_OT_6|GPIO_OTYPER_OT_7);

// to zero
SET_BIT(
    GPIOC->BRR,
    GPIO_BRR_BR_2|GPIO_BRR_BR_5|GPIO_BRR_BR_6|GPIO_BRR_BR_7);

// 4 th var: lines 0,1,3,4
// priorities 2(0), 3(2), 1(3), 2(2)
// groups/subgroups 4/4
// timer: TIM4 - priority 3(1)

// 4 bits for group and 4 bits for sub-group
NVIC_SetPriorityGrouping(5);
priGroup = NVIC_GetPriorityGrouping();
printf("Priority Group=%d\r\n",priGroup);

//
NVIC_SetPriority(EXTI0_IRQn, NVIC_EncodePriority(priGroup, 2, 0));
NVIC_DecodePriority(NVIC_GetPriority(EXTI0_IRQn),priGroup,&PreemptPriority,&SubPriority);
printf("EXTI0 Preempt Priority=%d \tSubPriority=%d\r\n",PreemptPriority,SubPriority);

NVIC_SetPriority(EXTI1_IRQn, NVIC_EncodePriority(priGroup, 3, 2));
NVIC_DecodePriority(NVIC_GetPriority(EXTI1_IRQn),priGroup,&PreemptPriority,&SubPriority);
printf("EXTI1 Preempt Priority=%d \tSubPriority=%d\r\n",PreemptPriority,SubPriority);

NVIC_SetPriority(EXTI3_IRQn, NVIC_EncodePriority(priGroup, 1, 3));
NVIC_DecodePriority(NVIC_GetPriority(EXTI3_IRQn),priGroup,&PreemptPriority,&SubPriority);
printf("EXTI3 Preempt Priority=%d \tSubPriority=%d\r\n",PreemptPriority,SubPriority);

NVIC_SetPriority(EXTI4_IRQn, NVIC_EncodePriority(priGroup, 2, 2));
NVIC_DecodePriority(NVIC_GetPriority(EXTI4_IRQn),priGroup,&PreemptPriority,&SubPriority);
printf("EXTI4 Preempt Priority=%d \tSubPriority=%d\r\n",PreemptPriority,SubPriority);
printf("Press any key\r\n");

// falling trigger on 0 1 3 4
SET_BIT(
    EXTI->FTSR,
    EXTI_FTSR_FT0|EXTI_FTSR_FT1|EXTI_FTSR_FT3|EXTI_FTSR_FT4);

// allow external interrupts
SET_BIT(
    EXTI->IMR,
    EXTI_IMR_IM0|EXTI_IMR_IM1|EXTI_IMR_IM3|EXTI_IMR_IM4);

//EXTI0=PC0 EXTI1=PC1 EXTI3=PC3 EXTI4=PC4

```

```

        SYSCFG->EXTICR[0]=SYSCFG_EXTICR1_EXTI0_PC|SYSCFG_EXTICR1_EXTI1_PC|SYSCFG_EXTICR1_EXTI3_PC;
        SYSCFG->EXTICR[1]=SYSCFG_EXTICR2_EXTI4_PC;
        NVIC_EnableIRQ(EXTI0_IRQn);
        NVIC_EnableIRQ(EXTI1_IRQn);
        NVIC_EnableIRQ(EXTI3_IRQn);
        NVIC_EnableIRQ(EXTI4_IRQn);

        SetupTIM4();

        while(1){
}

```

```

void TIM4_IRQHandler(void) {
    if (TIM4->SR & TIM_SR_UIF) {
        TIM4->SR &= ~TIM_SR_UIF;
        ITM_SendChar('o');
    }
}

```

```

void EXTI0_IRQHandler(void)
{
    EXTI->PR = EXTI_PR_PR0;
    ITM_SendChar('0');
    delay();
    ITM_SendChar('a');
    ITM_SendChar('\n');
}

```

```

void EXTI1_IRQHandler(void)
{
    EXTI->PR = EXTI_PR_PR1;
    ITM_SendChar('1');
    delay();
    ITM_SendChar('b');
    ITM_SendChar('\n');
}

```

```

void EXTI3_IRQHandler(void)
{
    EXTI->PR = EXTI_PR_PR3;
    ITM_SendChar('3');
    delay();
    ITM_SendChar('c');
    ITM_SendChar('\n');
}

```

```

void EXTI4_IRQHandler(void)
{
    EXTI->PR = EXTI_PR_PR4;
    ITM_SendChar('4');
    delay();
}

```

$$\}$$

Анализ результатов выполнения содержания работы

Результаты работы программы

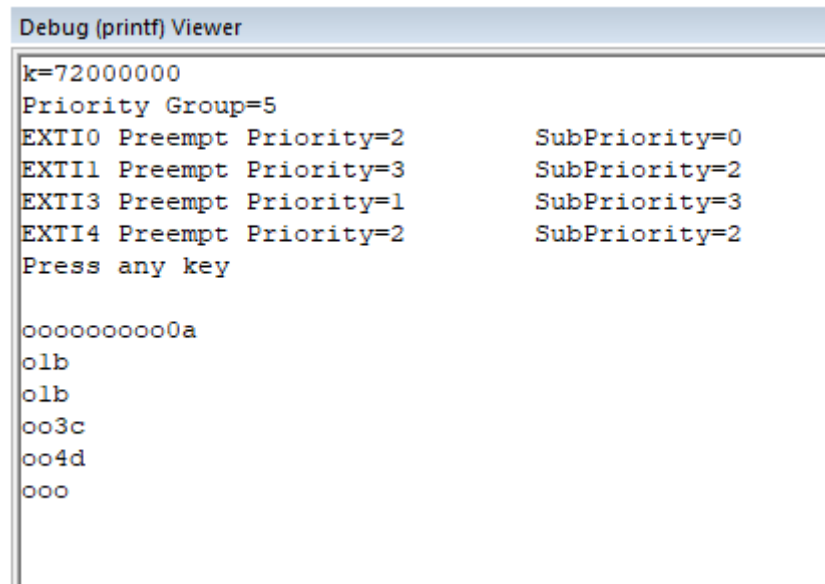


Рис. 1. Работа прерываний

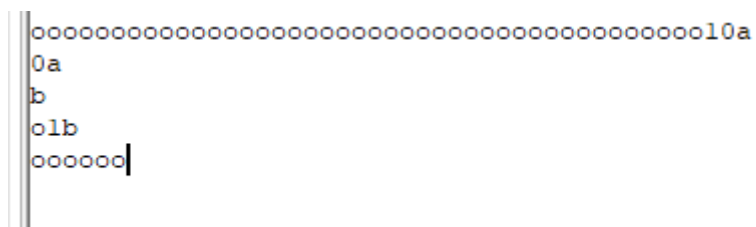


Рис. 2. Обработка прерываний из разных групп

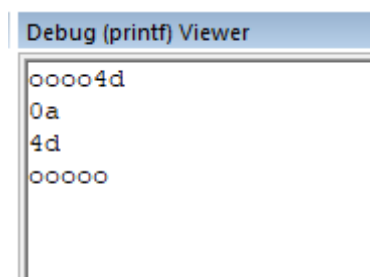


Рис.3 Обработка прерываний в одной группе в разных подгруппах

На рисунке 1 изображена последовательная обработка прерываний.

На рисунке 2 показано, что обработка прерывания с высшим приоритетом прерывает обработку прерывания с низшим приоритетом: приоритет по линии 1 – 3, по линии 0 – 2.

На рисунке 3 показано, что прерывания в одной группе не вытесняют друг друга

Выводы по точности измерения времени таймерами в зависимости от входящих прерываний.

Таймер TIM4 работает от аппаратного тактирования и “не сбивается” даже при обработке прерываний. Так как таймер имеет меньший приоритет, чем заданные прерывания, вывод таймера не вытесняет обработку прерываний.